How FFT Works

Talk – Term 3, Week 9

Brendan, Otto and Inuka

Outline

1 Housekeeping

2 FFT From A Mathematical View

3 FFT and Electrical Engineering

4 FFT Applications

5 Sampling is not an approximation!

6 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food



Outline

1 Housekeeping

2 FFT From A Mathematical View

3 FFT and Electrical Engineering

4 FFT Applications

5 Sampling is not an approximation!

6 Appendix and resources

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- `#define j i`

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.
- `#define N n`

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.
- Use N for the number of samples. In computer science, n is often used for this purpose, but we use N here.

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.
- Use N for the number of samples. In computer science, n is often used for this purpose, but we use N here.
- `#define n i`

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.
- Use N for the number of samples. In computer science, n is often used for this purpose, but we use N here.
- Index your array (data) with n instead of i, which is commonly used in computer science.

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.
- Use N for the number of samples. In computer science, n is often used for this purpose, but we use N here.
- Index your array (data) with n instead of i, which is commonly used in computer science.
- We use zero-based indexing.

Definitions and Notation

This talk is intended for audiences from multiple disciplines. Since notation can differ across fields, we will mostly follow electrical engineering conventions, while highlighting connections to mathematics and computer science.

- Refer to the complex number i as j.
- Use N for the number of samples. In computer science, n is often used for this purpose, but we use N here.
- Index your array (data) with n instead of i, which is commonly used in computer science.
- We use zero-based indexing.
- Data will be stored in an array x of N elements, where $x[n]$ denotes the nth element for $0 \leq n \leq N - 1$.

What Actually Is FFT?

What Actually Is FFT?

It's an efficient way to compute the Discrete Fourier Transform (DFT).

Ok, Fine, Then What Is the DFT?

Ok, Fine, Then What Is the DFT?

You start with x:

- **Computer science:** an array.
- **Electrical engineering:** sequence of a measured value over time (A SIGNAL).

Ok, Fine, Then What Is the DFT?

You start with x:

- **Computer science:** an array.
- **Electrical engineering:** sequence of a measured value over time (A SIGNAL).

The Discrete Fourier Transform (DFT) produces a **new sequence** $\hat{X}$.

- You can tell this is important because now the x is big and has a hat!

Precise Definition of DFT

Definition

Discrete Fourier Transform (DFT)

$$\hat{X}[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi knj}{N}},$$

where N is the sample size and x is an array containing the sampled values of a signal.

Precise Definition of DFT

Definition

Discrete Fourier Transform (DFT)

$$\hat{X}[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi knj}{N}},$$

where N is the sample size and x is an array containing the sampled values of a signal.

Take the polynomial $p(t) = \sum_{n=0}^{N-1} x[n] t^n$. Then $\hat{X}[k] = p\left(e^{-\frac{2\pi kj}{N}}\right)$.

Precise Definition of DFT

Definition

Discrete Fourier Transform (DFT)

$$\hat{X}[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi knj}{N}},$$

where N is the sample size and x is an array containing the sampled values of a signal.

Take the polynomial $p(t) = \sum_{n=0}^{N-1} x[n] t^n$. Then $\hat{X}[k] = p\left(e^{-\frac{2\pi kj}{N}}\right)$.

That is, the DFT is the output obtained by evaluating the polynomial at the values $e^{-\frac{2\pi kj}{N}}$, where $k = 0, 1, \ldots, N-1$ (the Nth roots of unity).

Precise Definition of DFT (In Electrical Engineering Terms)

Definition

Discrete Fourier Transform (DFT)

$$\hat{X}[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi knj}{N}},$$

where N is the sample size and x is an array containing the sampled values of a signal.

Define the 'z-transform' as the conversion of our signal $x[n]$ into a polynomial $X(z^{-1}) = \sum_{n=0}^{N-1} x[n] z^{-n}$, contextualising the delay unit z. Then $\hat{X}[k] = X\left(e^{\frac{2\pi kj}{N}}\right)$ converts into the digital frequency domain.

One Way to Think About the DFT (Electrical Engineering)

The DFT provides a discrete approximation of the frequency content of a measured signal.

Continuous Fourier Transform

Define the **Continuous Fourier Transform** of an integrable function f by

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) e^{-2\pi t \omega j} \, dt.$$

One Way to Think About the DFT (Electrical Engineering)

The DFT provides a discrete approximation of the frequency content of a measured signal.

Continuous Fourier Transform

Define the **Continuous Fourier Transform** of an integrable function f by

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) e^{-2\pi t \omega j} \, dt.$$

Fact: The Continuous Fourier Transform is important

One Way to Think About the DFT (Electrical Engineering)

The DFT provides a discrete approximation of the frequency content of a measured signal.

Continuous Fourier Transform

Define the **Continuous Fourier Transform** of an integrable function f by

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) e^{-2\pi t \omega j} \, dt.$$

Take samples of f to obtain x. Then the transform can be approximated in the form of

$$\sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi k n j}{N}},$$

as a function of k.

Trust Me

In a purely mathematical sense, the DFT is useful in combinatorics when dealing with convolutions and polynomial multiplication. We will use this to motivate how the FFT works as well.

Trust Me

In a purely mathematical sense, the DFT is useful in combinatorics when dealing with convolutions and polynomial multiplication. We will use this to motivate how the FFT works as well.

> ***Fourier Transform*** – *"A systematic way to decompose generic functions into a superposition of symmetric functions."* – *Terry Tao*

Trust Me

In a purely mathematical sense, the DFT is useful in combinatorics when dealing with convolutions and polynomial multiplication. We will use this to motivate how the FFT works as well.

> ***Fourier Transform*** – *"A systematic way to decompose generic functions into a superposition of symmetric functions."* – *Terry Tao*

This is related to thinking of our data x as the coefficients of a polynomial. Taking the DFT of a polynomial makes multiplication easier, since the DFT is essentially evaluation at the roots of unity.

Sequences as Polynomials

Take the sequence $x = [3, 4, 2]$. We can associate it with the polynomial

$$p(t) := \sum_{n=0}^{2} x[n]t^n = 3 + 4t + 2t^2.$$

Sequences as Polynomials

Take the sequence $x = [3, 4, 2]$. We can associate it with the polynomial

$$p(t) := \sum_{n=0}^{2} x[n]t^n = 3 + 4t + 2t^2.$$

Now take $y = [5, 0, 3, 7]$, with

$$q(t) := \sum_{n=0}^{3} y[n]t^n = 5 + 3t^2 + 7t^3.$$

Question: What is the coefficient of t^3 in the product polynomial $q(t)p(t)$?
It is

$$\sum_{n=0}^{3} x[n]y[3-n] = 7 \cdot 3 + 3 \cdot 4 + 0 \cdot 2 = 33,$$

treating $x[3]$ as 0.

Multiplying Polynomials

In fact, if x and y are two sequences of length N, with associated polynomials

$$p(t) := \sum_{n=0}^{N-1} x[n]t^n, \qquad q(t) := \sum_{n=0}^{N-1} y[n]t^n,$$

then the coefficient of t^k in $q(t)p(t)$, for $0 \leq k \leq N-1$, is $\sum_{n=0}^{k} x[n]y[k-n]$.

Definition: Convolution

Given two sequences x and y, their convolution $x \star y$ is defined by

$$(x \star y)[k] = \sum_{n=0}^{k} x[n]y[k-n],$$

for $0 \leq k \leq N-1$.

Convolution Notes

Notes

- The convolution sequence has the same length as x and y under this definition.

Convolution Notes

Notes

- The convolution sequence has the same length as x and y under this definition.
- The convolution is the sequence of coefficients of the product polynomial of the two sequences... almost!

Convolution Notes

Notes

- The convolution sequence has the same length as x and y under this definition.
- The convolution is the sequence of coefficients of the product polynomial of the two sequences... almost!
- The issue is that this convolution only gives the first N coefficients of the product polynomial. How do we fix this?

Convolution Notes

Notes

- The convolution sequence has the same length as x and y under this definition.
- The convolution is the sequence of coefficients of the product polynomial of the two sequences... almost!
- The issue is that this convolution only gives the first N coefficients of the product polynomial. How do we fix this?

  Solution: Pad the original sequences with zeros. We need a length of at least $2N - 1$, since that is the number of coefficients in the product polynomial.

Convolution Example

Example

Take $x = [1, 5, 3, 0, 0]$ and $y = [2, 7, 4, 0, 0]$, which have been padded with zeros. Their associated polynomials are $p(t) = 1 + 5t + 3t^2$ and $q(t) = 2 + 7t + 4t^2$.

Convolution Example

Example

Take $x = [1, 5, 3, 0, 0]$ and $y = [2, 7, 4, 0, 0]$, which have been padded with zeros. Their associated polynomials are $p(t) = 1 + 5t + 3t^2$ and $q(t) = 2 + 7t + 4t^2$.
Then

$$(1 + 5t + 3t^2)(2 + 7t + 4t^2) = 2 + 17t + 45t^2 + 41t^3 + 12t^4.$$

Convolution Example

Example

Take $x = [1, 5, 3, 0, 0]$ and $y = [2, 7, 4, 0, 0]$, which have been padded with zeros. Their associated polynomials are $p(t) = 1 + 5t + 3t^2$ and $q(t) = 2 + 7t + 4t^2$.
Then

$$(1 + 5t + 3t^2)(2 + 7t + 4t^2) = 2 + 17t + 45t^2 + 41t^3 + 12t^4.$$

Hence,

$$x \star y = [2, 17, 45, 41, 12].$$

Uses of Convolutions

- They give the product polynomials associated with sequences, as we observed.

Uses of Convolutions

- They give the product polynomials associated with sequences, as we observed.
- They can compute moving averages of data.

Uses of Convolutions

- They give the product polynomials associated with sequences, as we observed.
- They can compute moving averages of data.
- They are useful for efficient computation in competitive programming.

Uses of Convolutions

- They give the product polynomials associated with sequences, as we observed.
- They can compute moving averages of data.
- They are useful for efficient computation in competitive programming.
- There is an analogous continuous version, $\int_{-\infty}^{\infty} f(x-t)g(t)\,dt$, which has applications in electrical engineering and statistics (e.g., sums of random variables).

DFT and Convolutions

Recall that $\hat{X}$ denotes the DFT of x. Hopefully, the previous slide showed you that convolution, a type of multiplication, is useful.

DFT and Convolutions

Recall that $\hat{X}$ denotes the DFT of x. Hopefully, the previous slide showed you that convolution, a type of multiplication, is useful.

The **DFT** is useful because it allows us to compute convolutions efficiently.

DFT of a Convolution

For sequences x and y, let $\odot$ denote pointwise multiplication. Then (with sufficient padding),

$$\widehat{(x \star y)} = \hat{x} \odot \hat{y}.$$

This says that convolution in one domain corresponds to pointwise multiplication in another domain, where multiplication is easy to compute.

Another Way to Multiply Polynomials

To justify the result on the previous slide, consider the following way to multiply polynomials by treating them as functions.

Another Way to Multiply Polynomials

To justify the result on the previous slide, consider the following way to multiply polynomials by treating them as functions.

- A polynomial of degree m is uniquely determined by its values at $m + 1$ distinct points.

Another Way to Multiply Polynomials

To justify the result on the previous slide, consider the following way to multiply polynomials by treating them as functions.

- A polynomial of degree m is uniquely determined by its values at $m+1$ distinct points.
- For example, $t^2 - 6t + 1$ is uniquely determined by its values 1 at $t = 0$, 8 at $t = -1$, and -4 at $t = 1$. See polynomial interpolation.

Another Way to Multiply Polynomials

To justify the result on the previous slide, consider the following way to multiply polynomials by treating them as functions.

- A polynomial of degree m is uniquely determined by its values at $m+1$ distinct points.
- For example, $t^2 - 6t + 1$ is uniquely determined by its values 1 at $t = 0$, 8 at $t = -1$, and -4 at $t = 1$. See polynomial interpolation.
- Evaluating the product $p(t)q(t)$ at a point gives the product of the evaluations: $p(t)q(t)$ evaluated at $t = a$ equals $p(a)q(a)$.

Another Way to Multiply Polynomials

To justify the result on the previous slide, consider the following way to multiply polynomials by treating them as functions.

- A polynomial of degree m is uniquely determined by its values at $m+1$ distinct points.
- For example, $t^2 - 6t + 1$ is uniquely determined by its values 1 at $t = 0$, 8 at $t = -1$, and -4 at $t = 1$. See polynomial interpolation.
- Evaluating the product $p(t)q(t)$ at a point gives the product of the evaluations: $p(t)q(t)$ evaluated at $t = a$ equals $p(a)q(a)$.
- Recall that the DFT evaluates a polynomial at specific points. Therefore, pointwise multiplication of $\hat{X}$ and $\hat{Y}$ gives the evaluations of the product polynomial. This is the key idea behind the convolution theorem.

Which Points?

We have established that evaluating polynomials at specific points lets us find their product through pointwise multiplication. But which points should we choose, and how long will this take?

Which Points?

We have established that evaluating polynomials at specific points lets us find their product through pointwise multiplication. But which points should we choose, and how long will this take?

- As discussed earlier, to recover the full convolution of sequences of length N, we need to pad them with zeros to a length of at least $2N - 1$.

Which Points?

We have established that evaluating polynomials at specific points lets us find their product through pointwise multiplication. But which points should we choose, and how long will this take?

- As discussed earlier, to recover the full convolution of sequences of length N, we need to pad them with zeros to a length of at least $2N - 1$.
- Evaluating a polynomial of degree $O(N)$ at $O(N)$ points naively takes how long?

Which Points?

We have established that evaluating polynomials at specific points lets us find their product through pointwise multiplication. But which points should we choose, and how long will this take?

- As discussed earlier, to recover the full convolution of sequences of length N, we need to pad them with zeros to a length of at least $2N - 1$.
- Evaluating a polynomial of degree $O(N)$ at $O(N)$ points naively takes how long?

  Complexity: Each evaluation takes $O(N)$ time, and there are $O(N)$ evaluations, giving $O(N^2)$ time in total. For example, evaluating $2 + 5t + 6t^2 + 4t^3 = 2 + t(5 + t(6 + 4t))$ at one point takes $O(N)$ time using Horner's method. How can we do better?

Which Points?

We have established that evaluating polynomials at specific points lets us find their product through pointwise multiplication. But which points should we choose, and how long will this take?

- As discussed earlier, to recover the full convolution of sequences of length N, we need to pad them with zeros to a length of at least $2N - 1$.
- Evaluating a polynomial of degree $O(N)$ at $O(N)$ points naively takes how long?

  Complexity: Each evaluation takes $O(N)$ time, and there are $O(N)$ evaluations, giving $O(N^2)$ time in total. For example, evaluating $2 + 5t + 6t^2 + 4t^3 = 2 + t(5 + t(6 + 4t))$ at one point takes $O(N)$ time using Horner's method. How can we do better?

  Solution: We can achieve $O(N \log N)$ time by choosing evaluation points with as much **symmetry** as possible, reducing repeated work.

An Example

The key is to split our polynomial into its even- and odd-degree terms. Write

$$p(t) = a(t^2) + tb(t^2).$$

Suppose we want to find $p(1)$ and $p(-1)$.

An Example

The key is to split our polynomial into its even- and odd-degree terms. Write

$$p(t) = a(t^2) + tb(t^2).$$

Suppose we want to find $p(1)$ and $p(-1)$.
Then we only need to find $a(1)$ and $b(1)$, and combine them to obtain

$$p(1) = a(1) + b(1), \qquad p(-1) = a(1) - b(1).$$

An Example

The key is to split our polynomial into its even- and odd-degree terms. Write

$$p(t) = a(t^2) + tb(t^2).$$

Suppose we want to find $p(1)$ and $p(-1)$.
Then we only need to find $a(1)$ and $b(1)$, and combine them to obtain

$$p(1) = a(1) + b(1), \qquad p(-1) = a(1) - b(1).$$

The useful observation is that $1^2 = (-1)^2$. But we want to evaluate at more than two points. We need multiple points with useful square-root properties.

Going Deeper

We would like a way to recurse on the evaluation points 1 and -1, finding points whose squares give these values.

Going Deeper

We would like a way to recurse on the evaluation points 1 and -1, finding points whose squares give these values.
Recall that $i^2 = (-i)^2 = -1$. Then

$$\begin{aligned} p(i) &= a(-1) + ib(-1), \\ p(-i) &= a(-1) - ib(-1), \\ p(1) &= a(1) + b(1), \\ p(-1) &= a(1) - b(1). \end{aligned}$$

Thus, evaluating p at four points reduces to evaluating a and b at two points, with some additional additions and multiplications.

Divide and Conquer

We repeatedly take square roots to generate our evaluation points.
Similar to how we pad our sequences with zeros to obtain a length of at least $2N - 1$, we can pad further to reach a power of two, say 2^k.

We then reduce the problem of evaluating one polynomial at 2^k points to evaluating two polynomials of roughly half the degree at 2^{k-1} points each.

The work to combine the results is $O(2^k)$.

How Good Is This?

Assuming this approach works, our algorithm takes time $T(N)$, where

$$T(N) = 2T\left(\frac{N}{2}\right) + O(N).$$

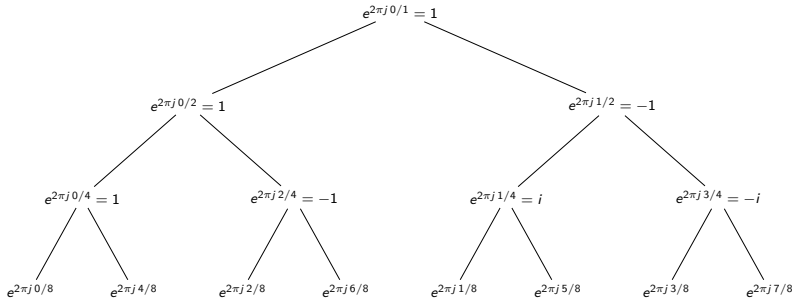
We divide the problem in half repeatedly, for $k = \log_2 N$ levels, until

$$\frac{N}{2^k} = 1.$$

This gives a total running time of $O(N \log N)$.

What Do These Numbers Look Like?

We evaluate at 1, then at 1 and -1, then at $1, -1, i, -i$, and continue down this tree.



The kth layer contains the 2^kth roots of unity. This is what we use.

Why Roots of Unity?

Definition: Roots of Unity

Define $\zeta_m = e^{\frac{2\pi j}{m}}$ as a primitive mth root of unity. The roots of unity are $\zeta_m^0, \zeta_m^1, \ldots, \zeta_m^{m-1}$, which are the solutions to the polynomial equation $z^m = 1$.

These are symmetrically distributed points around the unit circle. Recall that we defined the DFT as evaluating the associated polynomial at the roots of unity ($e^{-\frac{2\pi jk}{N}}$ is an Nth root of unity).

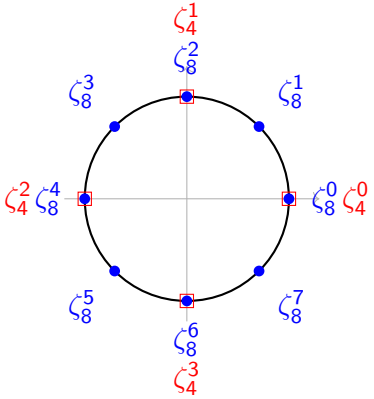
Visualising Roots of Unity

For m a power of two, an important observation is that

$$\left(\zeta_{2m}^{k}\right)^2 = \zeta_m^k.$$

In other words, squaring the $2m$th roots of unity gives the mth roots of unity.

For example, the squares of the 8th roots of unity give the 4th roots of unity, as illustrated on the right.



Wrapping It Up

With $p(t) = a(t^2) + tb(t^2)$, if we have already evaluated a and b at the mth roots of unity, then

$$p(\zeta_{2m}^k) = a(\zeta_m^k) + \zeta_{2m}^k b(\zeta_m^k).$$

This gives us an efficient way to combine the results after computing the DFTs of a and b, with $O(N)$ time to combine them.

Wrapping It Up

With $p(t) = a(t^2) + tb(t^2)$, if we have already evaluated a and b at the mth roots of unity, then

$$p(\zeta_{2m}^k) = a(\zeta_m^k) + \zeta_{2m}^k b(\zeta_m^k).$$

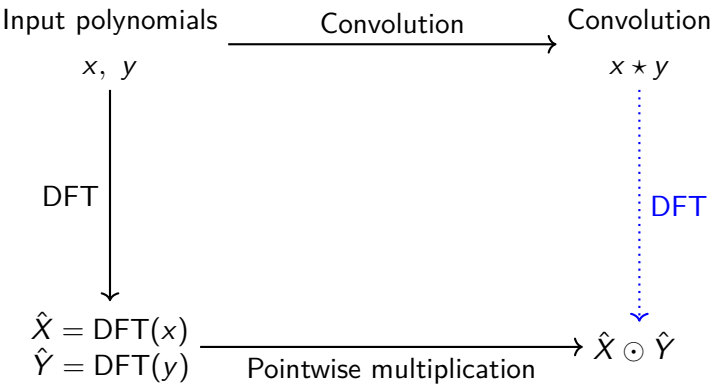
This gives us an efficient way to combine the results after computing the DFTs of a and b, with $O(N)$ time to combine them.

To summarise, we compute the DFT, which evaluates a polynomial at the $2m$th roots of unity (where m is a power of two), efficiently by evaluating its even and odd parts at the mth roots of unity and combining the results in $O(N)$ time.

This is a recursive divide-and-conquer algorithm. If the DFTs of a and b each take $O\left(\frac{N}{2} \log \frac{N}{2}\right)$ time, since each is a half-size problem, the total complexity is $O\left(2 \times \frac{N}{2} \log \frac{N}{2} + N\right) = O(N \log N)$, as claimed.

Inverse

Now we can find $\hat{X} \odot \hat{Y}$ in $O(N \log N)$ time, since pointwise multiplication takes $O(N)$ time. However, we need a way to reverse the DFT arrow below.



We Need a Hero

We want to compute the inverse DFT in $O(N \log N)$ time. Fortunately, this is very similar to computing the DFT: the inverse also involves evaluating a sequence at roots of unity, which we have already discussed. We won't delve into the details, but the relevant mathematical principle is the following symmetry:

- If α and β are Nth roots of unity, then β^{-1} and $\alpha\beta$ are also Nth roots of unity, and

$$\sum_{n=0}^{N-1} \left(\frac{\alpha}{\beta}\right)^n = \begin{cases} 0 & \text{if } \alpha \neq \beta, \\ N & \text{if } \alpha = \beta. \end{cases}$$

- This is related to the inverse of the Vandermonde matrix associated with the DFT.

(These are group properties.)

The Story

Recall that we took a step back to ask why the DFT uses roots of unity and what its purpose is. We considered our original sequence x, why computing its DFT efficiently matters for time complexity, and how this connects to applications in computer science.

The Story

Recall that we took a step back to ask why the DFT uses roots of unity and what its purpose is. We considered our original sequence x, why computing its DFT efficiently matters for time complexity, and how this connects to applications in computer science.

1. The FFT is important because it allows us to compute convolutions efficiently.

The Story

Recall that we took a step back to ask why the DFT uses roots of unity and what its purpose is. We considered our original sequence x, why computing its DFT efficiently matters for time complexity, and how this connects to applications in computer science.

1. The FFT is important because it allows us to compute convolutions efficiently.
2. This efficiency comes from the relationship between convolution and polynomial multiplication: polynomial multiplication becomes pointwise multiplication when evaluated at the roots of unity.

The Story

Recall that we took a step back to ask why the DFT uses roots of unity and what its purpose is. We considered our original sequence x, why computing its DFT efficiently matters for time complexity, and how this connects to applications in computer science.

1. The FFT is important because it allows us to compute convolutions efficiently.
2. This efficiency comes from the relationship between convolution and polynomial multiplication: polynomial multiplication becomes pointwise multiplication when evaluated at the roots of unity.
3. To compute the DFT, we evaluate a polynomial at the roots of unity. By splitting this into two smaller problems recursively, we achieve $O(N \log N)$ time.

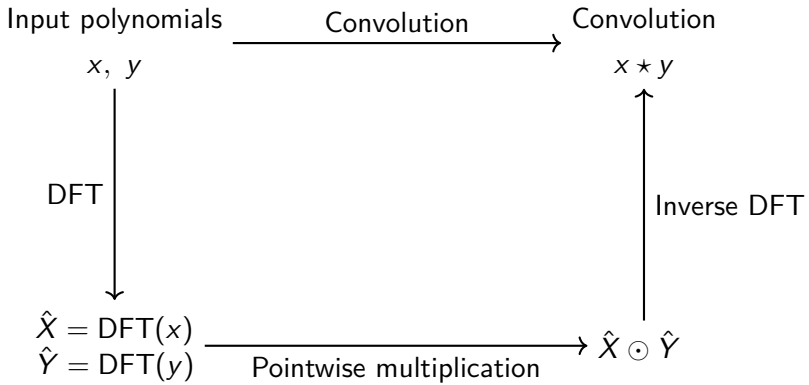
The Story

Recall that we took a step back to ask why the DFT uses roots of unity and what its purpose is. We considered our original sequence x, why computing its DFT efficiently matters for time complexity, and how this connects to applications in computer science.

1. The FFT is important because it allows us to compute convolutions efficiently.
2. This efficiency comes from the relationship between convolution and polynomial multiplication: polynomial multiplication becomes pointwise multiplication when evaluated at the roots of unity.
3. To compute the DFT, we evaluate a polynomial at the roots of unity. By splitting this into two smaller problems recursively, we achieve $O(N \log N)$ time.
4. The inverse DFT can be computed in a similar way to the DFT itself.

A (Commutative) Diagram

We summarise the key ideas in the following diagram:



Outline

1 Housekeeping

2 FFT From A Mathematical View

3 **FFT and Electrical Engineering**

4 FFT Applications

5 Sampling is not an approximation!

6 Appendix and resources

Recall the Frequency domain...

When analysing signals, we can characterise them by one of two primary descriptions...

What values do they take over time?

What frequencies is it composed of?

Where will an Electrical Engineer encounter the Frequency domain?

In UNSW, Electrical Engineering students are explicitly exposed to the frequency domain in...
ELEC2134 - Circuits and Signals
ELEC2133 - Analogue Electronics
ELEC3104 - Digital Signal Processing
ELEC3106 - Electroncis
TELE3113 - Introduction to Analogue and Digital Communications
And in further signal processing courses.

But why?

As mentioned previously, Electrical Engineering is closely tied to observing and manipulating signals, this concerns audible sounds, electrical signals, radio signals, physical quantities, image signals and much more.

Fundamentally, the study of signals leads to the study of filters. Our faviourite filters are filters that are linear and time-invariant (stay the same throughout time)

<insert marco polo shenanigans>

What is a Filter?

Everything is a filter...

A filter is a process that amplifies or reduces components of an input to produce an output.

An easy example is how the average of a dataset is a filter, it keeps the DC (average) value but removes all other frequency components (anything that varies across the dataset)

How do we get to the FFT?

As much as we love the Fourier Transform (FT) as our window into the frequency domain, it requires calculations over a continuous domain to obtain, digital computers cannot do this.
<image of fourier transform>

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt$$

Given this limitation we can sample our continuous time signal into a discrete time signal and apply what we called a Discrete Time Fourier Transform (DTFT) to give us an approximate result of the FT:

$$X(\theta) = X(z)|_{z=e^{-j\theta}} = \sum_{n=0}^{N-1} x[n] e^{-jn\theta}$$

(yes this notation is confusing) where we use theta to denote a digital frequency (from 0 to 2π) as this conversion into the discrete time domain removes the context of a sampling rate ($n = t \times fs$ or $n \times T_s = t$).

How do we get to the FFT?

But now the challenge lies in how we realise this continuous frequency domain... We do it through the same process of discrete approximation, bringing us to the Discrete Fourier Transform (DFT)

$$\hat{X}[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi nk/N}$$

This is where the number of frequency 'taps' is equal to the length of the input signal.

Approximately?

The term of approximation has been used somewhat loosely in our definitions of the DTFT and the DFT. This is as our discretisation of our time and frequency domains has led to what can be known as a bandlimited or resolution limited representation.
to conceptualise, consider 2 things

1. Depending on the sampling rate what information can we keep, what information is lost, and what information is ambiguous?
2. Can I tell what the main frequency is in this spectra? <image of spectra with a peak at two adjacent points>

Aliasing, Nyquist, Reconstruction, Modulation, and Resolution

These two considerations open up a variety of topics:

1. Aliasing and Nyquist -> what frequency a signal looks like vs what it was.
2. Reconstruction -> sinc is the bridge between continuous and discrete time domains
3. Resolution -> To have more detailed DFT results, we need more detail data, we need MORE SAMPLES

Outline

1 Housekeeping

2 FFT From A Mathematical View

3 FFT and Electrical Engineering

4 **FFT Applications**

5 Sampling is not an approximation!

6 Appendix and resources

Spectrum Analysers

In areas such as radio, safety, circuits, and real-time systems, A common requirement is the analysis of signals to identify when a specific frequency or set of frequencies are present and above a defined threshold. The common limiting factor in the performance of solutions in this task is the sampling rate against the required accuracy of time to define, and the accuracy of the frequencies to be observed.

A common approach to these problems is a method called the short-time fourier transform (STFT), this is where the FFT of batches of data is computed to analyse frequency changes over time.

Example 1

For a signal that is sampled at 20kHz, a system processes batches of 'N' samples every 100ms for an fft of the past 100ms, find what the time resolution and frequency resolution of the solution is.

Solution

Solution 1

for a 20kHz signal, we have 2000 samples being processed every 100ms frame. so our STFT can comfortably produce a 10Hz frequency resolution fft with a time resolution of 100ms.

Note here that if we double the time per frame (200ms time resolution), we double the frequency resolution to 5Hz. So we can recognise a fundamental trade in uncertainties:

$$\frac{f}{\Delta f} = N = f\Delta t$$

$$\Delta f \Delta t = 1$$

Audio Processing

From audio effects to instrument tuning, The goal of some microphone digital systems is to ensure that the sound captured by the microphone sensor and is amplified by the speaker systems doesn't cause ringing (the high pitched squealing when a microphone is next to a speaker).

Example 2

For a microphone-speaker system, it is known that an impulse input to the microphone will produce an output (heard by the microphone) with the following fft: <image of a magnitude and phase response>

Design what the fft of the filter to be placed just before the speaker output should look like with respect to it's magnitude and phase responses.

Solution

Solution 2

Here it should be noted that if an input to a microphone is outputting to a speaker that is being heard again by the same microphone, then we have a system with closed loop feedback, this means that all sounds will bounce infinitely from the microphone from the speakers, meaning our cascaded system filter and the impulse description need to result in all output points of our fft being inside the unit circle.

This delves into a portion of digital control theory for first order systems, however we can communicate this in terms of the divergence and convergence of infinite geometry series.

So we require the product of our two magnitude graphs to be inside the unit circle. (this is assuming it takes time for the microphone to pick up the reverb from the speaker.)

Signal Integrity and High frequency Signals

As mentioned previously, Basically everything is a filter, but in high speed electronics, this means everything is trying to reduce your electrical signals before they get to their destinations.

Example 3

When debugging a high speed data line in his PCB, Jack finds that when we switches the input from 0 to 1 (step input), the output fluctuates with the sequence [0, 1.8, 0.5, 1.2, 0.75, 1] and stays at 1. Jack wants to solve this problem using a filter at the output that when used, will convert this sequence back into [0, 1]. design this filter.

Solution

Solution 3

The goal of this example is to design an inverse filter, what we can notice is that we have an initial input to the system $[0, 1, 1, 1, 1, 1, \dots]$ which we convolute with x_1 to give us the sequence: $[0, 1.8, 0.5, 1.2, 0.75, 1]$. reversing this convolution we know this input filter follows: $[1.8, -1.3, 0.7, -0.45, 0.25]$.

A quick trick we can use to find the inverse would be to apply the inverse z transform, otherwise, we can do the reverse calculations, or even use the fft to find the inverse (equivalent methods)

Outline

1 Housekeeping

2 FFT From A Mathematical View

3 FFT and Electrical Engineering

4 FFT Applications

5 **Sampling is not an approximation!**

6 Appendix and resources

Shifted delta function δ_i

Delta Function

Define the delta function as

$$\delta[n] = \begin{cases} 1 & n = 0 \\ 0 & n \neq 0 \end{cases}$$

Shifted delta function δ_i

Delta Function

Define the delta function as

$$\delta[n] = \begin{cases} 1 & n = 0 \\ 0 & n \neq 0 \end{cases}$$

Shifted Delta Function

Define the shifted delta function as

$$\delta_i[n] = \delta[n - i]$$

Shifted delta function δ_i

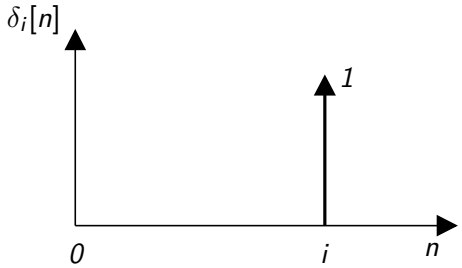


Figure: The shifted delta function is 0 everywhere except at $n = i$ where it equals 1.

Representing Samples

When we sample a signal, we are really evaluating a continuous underlying function.

Representing Samples

When we sample a signal, we are really evaluating a continuous underlying function.

$$x[n] = f(nT_s)$$

Where T_s is the sampling period, and $f_s = \frac{1}{T_s}$ is the sampling frequency. Note $n \in \mathbb{Z}$.

Representing Samples

When we sample a signal, we are really evaluating a continuous underlying function.

$$x[n] = f(nT_s)$$

Where T_s is the sampling period, and $f_s = \frac{1}{T_s}$ is the sampling frequency. Note $n \in \mathbb{Z}$.
We can think of our signal as an array of elements, kind of like a vector. We multiply each sample value by delta,

$$\mathbf{x} = \sum_{i=-\infty}^{\infty} x[i]\delta_i$$

Discrete signals are impulse trains

We can think of our signal as an array of elements, kind of like a weighted sum of basis vectors. We multiply each sample value by delta,

$$\mathbf{x} = \sum_{i=-\infty}^{\infty} x[i]\delta_i$$

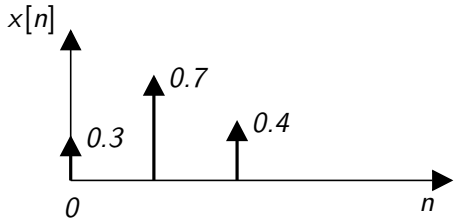


Figure: Signals can be represented as a weighted sum of deltas

Fourier Transform

Recall the definition of the continuous fourier transform (hereby refered to as the fourier transform),

Fourier Transform and Inverse

Define the **Fourier Transform** of a square integrable function f by

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} \, \mathrm{d}t \,,$$
$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{j\omega t} \, \mathrm{d}t.$$

Discrete Time Fourier Transform

Discrete Time Fourier Transform

Define the **Discrete Time Fourier Transform** of a sequence **x** as

$$\hat{x}(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}.$$

(we don't care about the inverse for this)

Commutative Diagram

In general, we have the following commutative diagram,

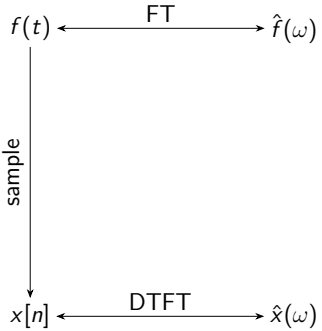


Figure: Commutative diagram. Take note of the arrows.

Aliasing

Nyquist Band-Limited Signal (N.B.L)

A signal $f(t)$ is N.B.L if it is band-limited, and it satisfies the nyquist criterion.

$$\hat{f}(\omega) = 0 \qquad \text{for } \omega \notin [-\pi, \pi], \tag{1}$$

1. Band-limited means the signal has a minimum and maximum frequency. I.e. its frequency content is limited to a certain band of frequencies.
2. The Nyquist criterion says that the sampling frequency is double the highest frequency present.

video (courtesy of claude)

Commutative Diagram

If our signal $f(t)$ is specifically Nyquist band limited (NBL), then we have this!

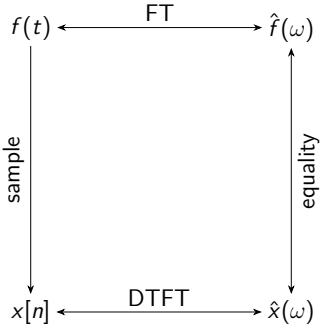


Figure: Commutative diagram. Take note of the arrows directions.

Desmos.

Sinc Fourier Transform

Suppose we have a signal $f(t)$, defined as,

$$f(t) = \text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}.$$

Sinc Fourier Transform

Suppose we have a signal $f(t)$, defined as,

$$f(t) = \text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}.$$

We can show that the fourier transform is

$$\hat{f}(\omega) = \text{rect}_{\pi}(\omega) = \begin{cases} 1 & |\omega| \leq \pi \\ 0 & \text{otherwise} \end{cases}$$

Sinc Fourier Transform

Suppose we have a signal $f(t)$, defined as,

$$f(t) = \text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}.$$

We can show that the fourier transform is

$$\hat{f}(\omega) = \text{rect}_{\pi}(\omega) = \begin{cases} 1 & |\omega| \leq \pi \\ 0 & \text{otherwise} \end{cases}$$

* I may be off by some constant factors, but it doesn't really matter.

Sinc Interpolation

Given a discrete signal $\mathbf{x}$, can we get back to $f(t)$?

Sinc Interpolation

Given a discrete signal $\mathbf{x}$, can we get back to $f(t)$?

- Because the signal is N.B.L, we can find $\hat{f}(\omega)$ like so,

$$\hat{f}(\omega) = \hat{x}(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

Sinc Interpolation

Given a discrete signal $\mathbf{x}$, can we get back to $f(t)$?

- Because the signal is N.B.L, we can find $\hat{f}(\omega)$ like so,

$$\hat{f}(\omega) = \hat{x}(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

- We can then take the inverse fourier transform

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \left(\sum_{-\infty}^{\infty} x[n] \delta_i \right) e^{j\omega t} \, \mathrm{d}\omega$$

Sinc Interpolation

Given a discrete signal $\mathbf{x}$, can we get back to $f(t)$?

- Because the signal is N.B.L, we can find $\hat{f}(\omega)$ like so,

$$\hat{f}(\omega) = \hat{x}(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

- We can then take the inverse fourier transform

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \left(\sum_{-\infty}^{\infty} x[n] \delta_i \right) e^{j\omega t} \, \mathrm{d}\omega$$

- (Note to self. Now finish the derivation on the board)

Outline

1 Housekeeping

2 FFT From A Mathematical View

3 FFT and Electrical Engineering

4 FFT Applications

5 Sampling is not an approximation!

6 **Appendix and resources**

Feedback!

Please fill out feedback form so we know how to run future workshops :D



(a) Feedback form



(b) Attendance form

Photo time :)